Slide Sebelumnya	←
Mode Layar Penuh (Fullscreen)	F
Speaker Notes View (Catatan Dosen)	S
Slide Overview (Matriks)	0 / Esc
Blackout / Layar Hitam (Fokus Diskusi)	B / .

Jaringan Komputer

Pertemuan 05 • Lapisan Transport

DOSEN PENGAMPU

Rajif Agung Yunmar, S.Kom., M.Cs.

I Wayan Wiprayoga Wisesa, S.Kom., M.Kom.

Hartanto Tantriawan, S.Kom., M.Kom.

Ilham Firman Ashari, S.Kom., M.T.

Program Studi Teknik Informatika · Fakultas Teknologi Industri · Institut Teknologi Sumatera (ITERA)

Outline Materi

Bagian 1: Konsep Transport & UDP

- 1. Layanan Lapisan Transport
- 2. Multiplexing & Demultiplexing
- 3. User Datagram Protocol (UDP)
- 4. Prinsip Reliable Data Transfer (RDT)

Bagian 2: TCP & Protokol Modern

- 5. Transmission Control Protocol (TCP)
- 6. Manajemen Koneksi TCP
- 7. Flow Control (Kendali Aliran)
- 8. TCP Congestion Control
- 9. Evolusi Transport Layer (QUIC)

1. Layanan Lapisan Transport

Peran Transport Layer

Menyediakan komunikasi **logis ujung-ke-ujung (end-to-end)** antara proses aplikasi yang berjalan di host yang berbeda.

- **Network Layer:** Komunikasi logis antar **Host** (komputer).
- **Transport Layer:** Komunikasi logis antar **Proses** (aplikasi/socket).

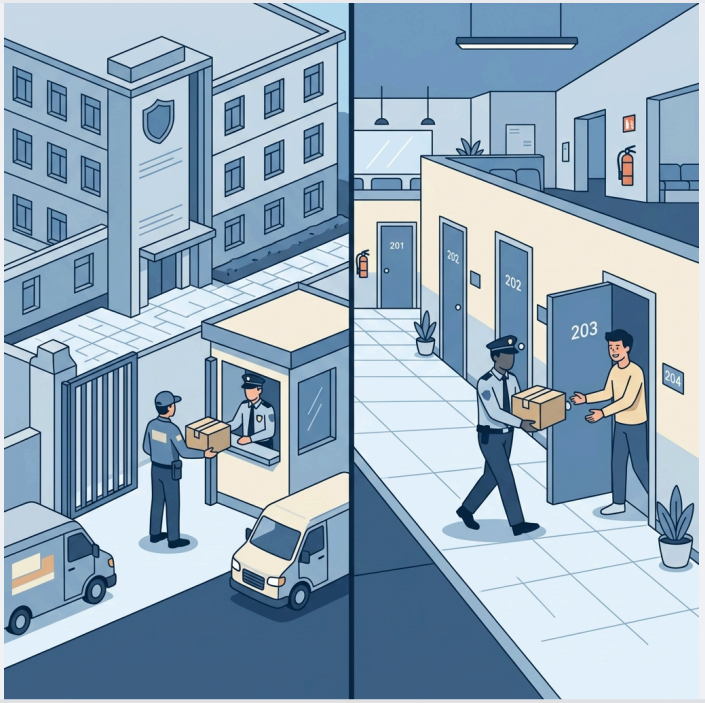
> 💡 **Analogi Asrama Mahasiswa (Kost-kostan):**
> Gedung Asrama = **Host** (Komputer/HP)
> Mahasiswa di kamar = **Proses** (Aplikasi: Chrome, Spotify, Zoom)
> Kurir Paket = **Network Layer** (Hanya mengantar paket sampai ke pos satpam asrama)
> Satpam Asrama = **Transport Layer** (Mensortir dan menyalurkan paket ke kamar mahasiswa yang tepat)

Ilustrasi Peran Transport Layer

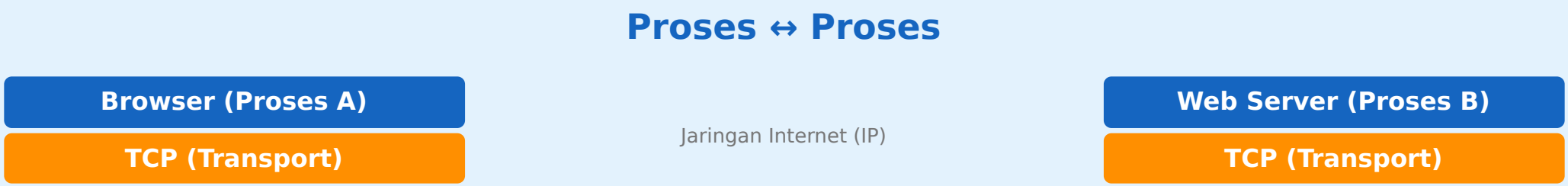
Analogi Asrama Mahasiswa

Bagian Kiri (Network Layer):
Kurir paket JNE hanya bertugas mengantar barang melalui jaringan jalan umum sampai ke **Pos Gerbang Asrama (Host)**.

Bagian Kanan (Transport Layer):
Satpam di dalam gedung asrama bertugas menyortir paket dan mengantarkannya langsung ke **pintu kamar mahasiswa (Proses/Aplikasi)** yang bersangkutan.

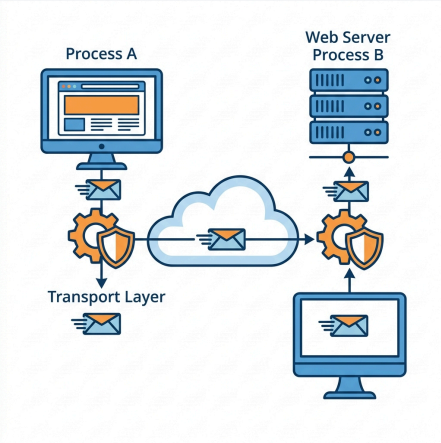


Komunikasi Proses ke Proses



Tugas utama transport layer adalah mengambil pesan dari aplikasi, memecahnya menjadi segmen-segmen kecil (jika perlu), dan menyerahkannya ke network layer. Di sisi penerima, menyusunnya kembali dan menyerahkannya ke aplikasi yang tepat.

Ilustrasi: Komunikasi Proses ke Proses



Transport layer mengambil pesan dari aplikasi (misal: Browser), menyerahkannya ke network layer, dikirim melalui internet. Di sisi penerima, pesan diserahkan kembali ke aplikasi yang tepat (misal: Web Server).

2. Multiplexing & Demultiplexing

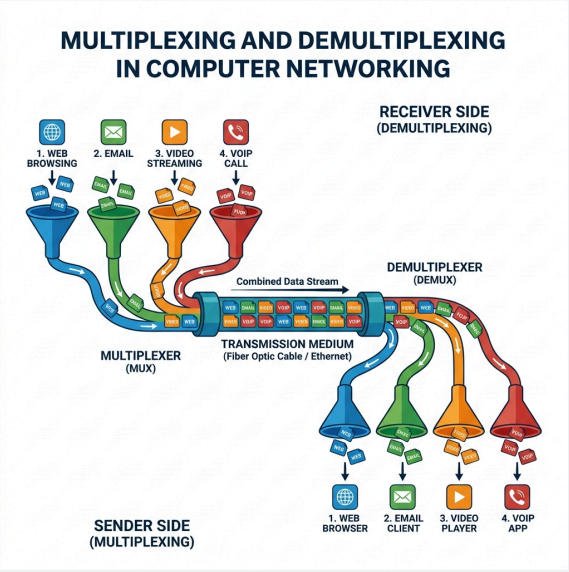
Konsep Mux & Demux

Di Sisi Pengirim (Multiplexing)

Mengumpulkan data dari banyak soket (aplikasi), menambahkan header transport (yang nantinya dipakai untuk demultiplexing), dan meneruskannya ke Network Layer lewat satu saluran.

Di Sisi Penerima (Demultiplexing)

Menggunakan informasi header (**Port Numbers**) untuk memilah dan mengantarkan segmen yang masuk ke soket yang tepat.



Demultiplexing: UDP

Connectionless Demux (UDP)

- Soket UDP diidentifikasi sepenuhnya oleh **2-tuple**: `(Dest IP, Dest Port)`.
- Jika dua datagram dari IP pengirim (sumber) yang berbeda, atau dari Port sumber yang berbeda, menuju **Port Tujuan yang sama**, keduanya akan masuk ke **satu soket UDP yang sama**.
- Sangat efisien untuk server yang melayani banyak klien sekaligus tanpa perlu melacak status tiap klien (contoh: DNS Server di Port 53).

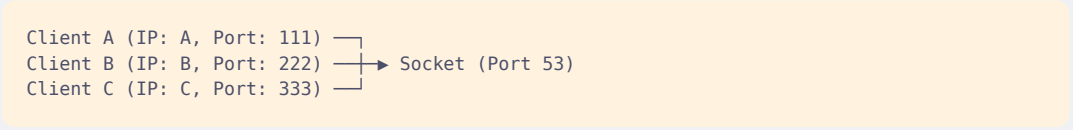
Demultiplexing: TCP

Connection-Oriented Demux (TCP)

- Soket TCP diidentifikasi secara unik oleh **4-tuple**: `(Src IP, Src Port, Dest IP, Dest Port)`.
- Host penerima menggunakan *keempat* nilai ini untuk mengarahkan segmen ke soket yang tepat.
- Web server (Apache/Nginx) membuat **soket baru yang terpisah** untuk *setiap* koneksi klien yang masuk, meskipun semuanya menuju port tujuan yang sama (Port 80/443).

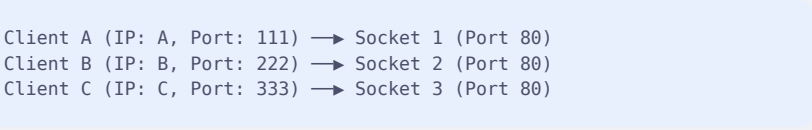
Perbandingan Demux UDP vs TCP

UDP Demux



Satu pintu (soket) melayani semua tamu yang datang ke alamat tersebut.

TCP Demux

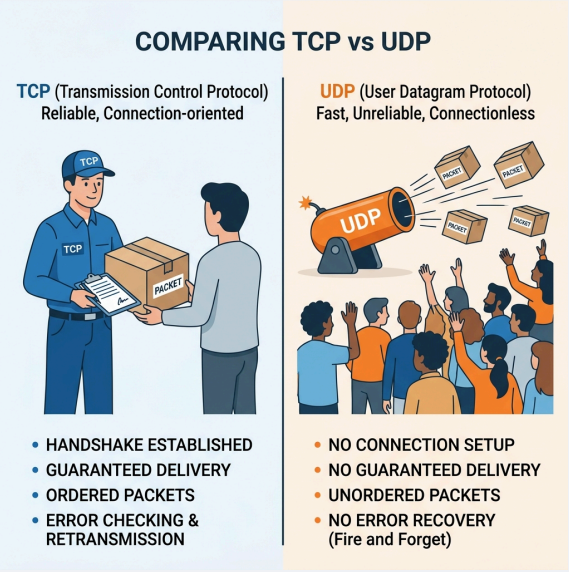


Setiap tamu (klien) dilayani di ruangan (soket) khusus yang berbeda, meskipun datang ke gedung (port) yang sama.

3. User Datagram Protocol (UDP)

UDP: Cepat Tapi Tidak Andal

- **No Frills, Bare Bones:** Protokol transport yang sangat minimalis (RFC 768).
- **Connectionless:** Tidak ada proses handshaking sebelum mengirim data. Langsung tembak!
- **Best Effort:** Segmen UDP dapat hilang (lost) atau datang tidak berurutan (out of order).
- **No Congestion Control:** UDP tidak peduli jaringan sedang macet, ia akan terus mengirim data.



Mengapa UDP Masih Digunakan?

Meskipun tidak andal, UDP memiliki keunggulan telak dalam skenario tertentu:

- **Tanpa delay setup:** Tidak perlu menunggu 3-way handshake selesai untuk mulai mengirim data (Sangat penting untuk DNS query yang butuh respon instan).
- **Tanpa state koneksi:** Server tidak perlu menyimpan informasi buffer atau parameter koneksi klien, sehingga memori jauh lebih hemat.
- **Header sangat kecil:** Hanya 8 byte overhead (dibandingkan TCP yang 20 byte).
- **Kontrol penuh di level aplikasi:** Aplikasi bisa mengontrol kapan tepatnya data dikirim tanpa dihalangi oleh algoritma congestion control TCP.

Pentingnya UDP:

Real-time & Low Latency



Dalam aplikasi real-time, **kecepatan lebih penting daripada keutuhan 100%**.

- 🎮 **Game Online:** Kehilangan satu paket data pergerakan pemain tidak masalah, asalkan paket berikutnya datang sangat cepat tanpa delay. Jika menggunakan TCP, retransmisi akan menyebabkan *lag/stuttering*.
- 📺 **Live Streaming / VoIP:** Hilangnya paket hanya menyebabkan sedikit glitch visual/audio sementara, yang lebih bisa ditoleransi manusia daripada video yang terus-terusan berhenti untuk *buffering*.

UDP Checksum

Fungsi Checksum

- Meskipun UDP tidak andal, ia menyediakan fungsi **deteksi error** minimum melalui *Checksum*.
- **Tujuan:** Mendeteksi "bit yang berubah" (error) di dalam segmen yang ditransmisikan.

Sisi Pengirim

- Memperlakukan segmen (termasuk header dan data) sebagai urutan integer 16-bit.
- **Checksum:** Hasil penjumlahan seluruh 16-bit word menggunakan *1's complement arithmetic*, kemudian dibalik (complement).
- Nilai ini dimasukkan ke field *checksum* di header UDP.

Sisi Penerima

- Penerima menjumlahkan semua 16-bit word yang diterima beserta checksum-nya.
- Jika tidak ada error, hasil penjumlahan (1's complement) harus berupa kumpulan bit `1` (semuanya bernilai 1).
- Jika ada setidaknya satu bit `0`, maka **error terdeteksi**, dan segmen UDP biasanya akan dibuang (di-drop).

> **Keterbatasan:** Checksum UDP lemah dibandingkan CRC pada Data Link Layer, namun sangat cepat dihitung dalam perangkat lunak (software).

4. Prinsip Reliable Data Transfer (RDT)

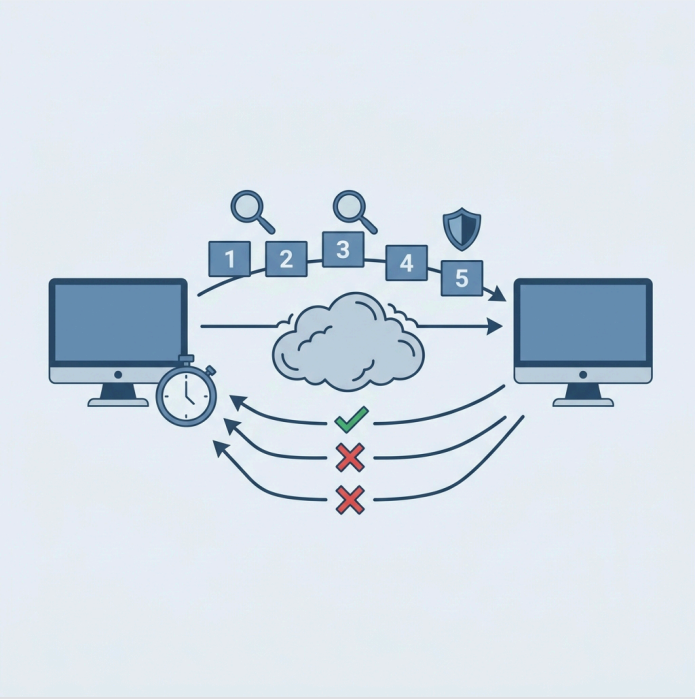
Reliable Data Transfer (RDT)

Tantangan Utama:

Bagaimana membangun layanan komunikasi yang **andal** (tidak ada data hilang, rusak, atau tertukar urutannya) di atas saluran fisik/jaringan yang **tidak andal** (unreliable channel)?

Mekanisme Dasar RDT

- **Error Detection:** Checksum (mendeteksi bit error).
- **Receiver Feedback:**
  - **ACK** (Acknowledgement): Berhasil diterima.
  - **NAK** (Negative ACK): Gagal/Rusak.
- **Retransmission:** Mengirim ulang paket yang rusak/hilang.
- **Sequence Number:** Menandai urutan paket agar penerima tahu jika ada paket duplikat atau hilang.
- **Timer:** Untuk mendeteksi paket yang hilang di jalan (Timeout).



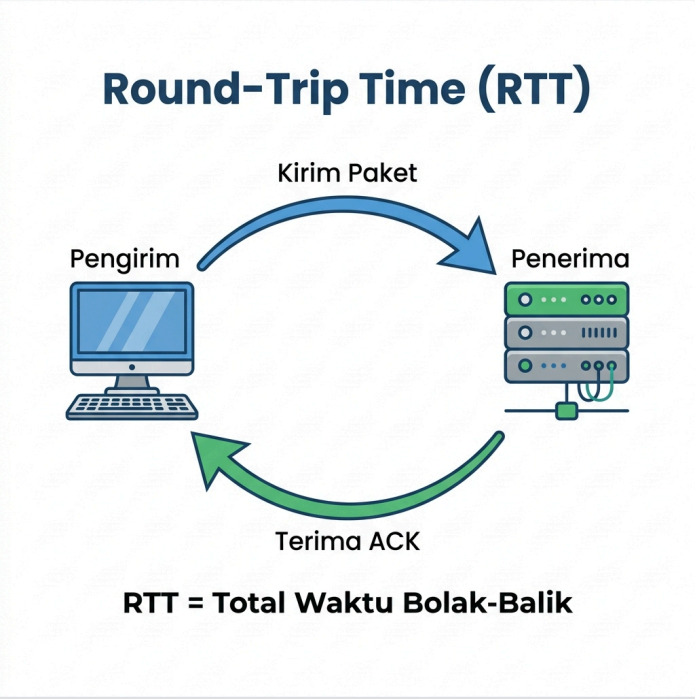
Ilustrasi Mekanisme RDT (Timer, Sequence, Feedback, & Error Detection)

Apa itu RTT (Round-Trip Time)?

Round-Trip Time (RTT) adalah total waktu yang dibutuhkan oleh sebuah paket data untuk melakukan perjalanan dari pengirim menuju penerima, ditambah dengan waktu kembalinya sinyal konfirmasi (ACK) ke pengirim.

Singkatnya: **Waktu sejak paket dikirim sampai konfirmasi diterima.**

- RTT sangat dipengaruhi oleh jarak fisik dan panjang kabel penghubung.
- Turut dipengaruhi oleh waktu antrean pemrosesan di router (Queueing Delay).
- **Mengapa Penting?** Nilai RTT sangat menentukan seberapa efisien kapasitas bandwidth bisa dimanfaatkan.



RTT = Total Waktu Bolak-Balik Paket dan ACK

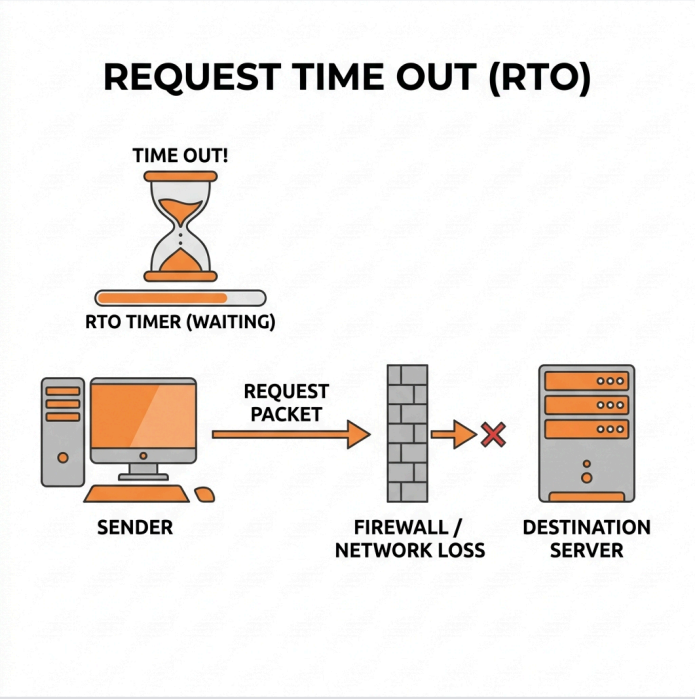
Studi Kasus: Request Time Out (RTO)

Apa yang terjadi?

Pengirim berhasil melempar paket ke jaringan, dan paket tersebut sedang mengalir mencari jalan. Namun, hingga *timer* (waktu tunggu) habis, tidak ada balasan sama sekali yang kembali ke pengirim.

Penyebab Umum:

- Paket ditolak secara diam-diam (di-drop) oleh **Firewall** di sisi penerima atau router tengah.
- Jaringan sangat macet (Congestion), sehingga paket terbuang dari antrean (Packet Loss).
- Server tujuan sedang mati/hang sehingga tidak merespon paket yang datang.



Waktu tunggu (timer) habis sebelum balasan tiba

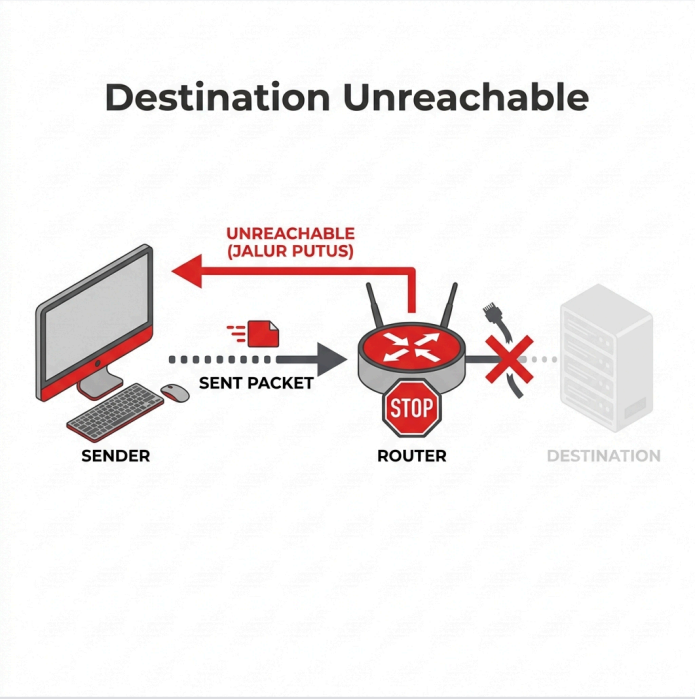
Studi Kasus: Destination Unreachable

Apa yang terjadi?

Paket bahkan tidak pernah sampai mendekati tujuan. Dalam perjalanannya, sebuah Router (atau PC pengirim itu sendiri) menyadari bahwa jalan menuju ke sana **tidak ada** atau terputus, lalu secara aktif mengirimkan pesan eror (ICMP) kembali ke pengirim.

Penyebab Umum:

- Kabel jaringan putus atau interface jaringan mati (Link Down).
- Tidak ada entri di tabel routing (No Route to Host).
- Salah konfigurasi IP atau Gateway.



Router mengirim pesan eror eksplisit: Jalur Putus

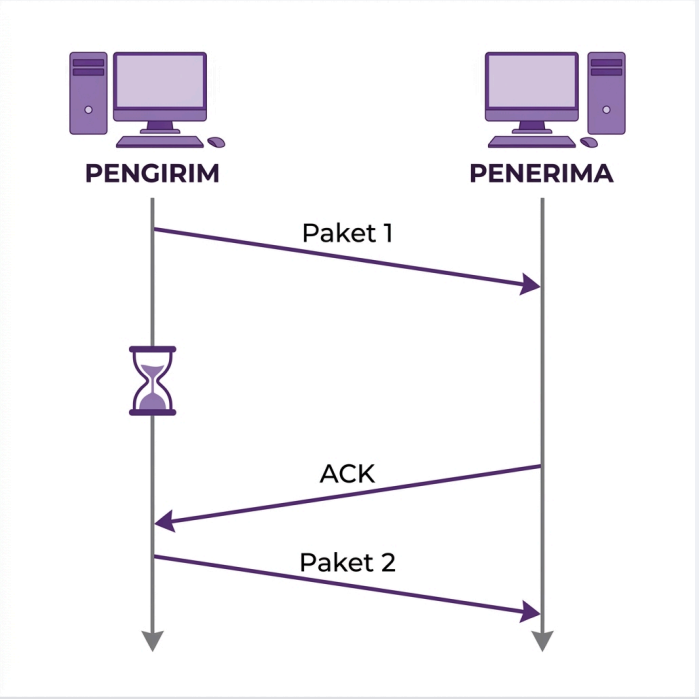
Perbandingan: RTO vs Unreachable

Aspek	Request Time Out (RTO)	Destination Unreachable
Status Jalan	Rute jalannya ada, tapi paket "hilang" di jalan atau tujuan.	Rute jalannya benar-benar tidak diketahui/terputus.
Respons Jaringan	**Diam (Silent Drop).** Tidak ada info apa-apa yang kembali.	**Aktif (Eksplisit).** Router mengirim notifikasi eror ICMP.
Analogi Kehidupan	Kirim barang lewat kurir, tapi kurirnya lenyap di jalan. Kita menunggu tanpa kepastian sampai bosan.	Kirim barang lewat kurir, tapi baru sampai gang depan ditolak satpam: "Alamat ini tidak ada!".

Evolusi RDT & Pipelining (1)

Pendekatan "Stop-and-Wait" (rdt3.0)

- Pengirim mengirim 1 paket, lalu **menunggu** ACK dari penerima sebelum mengirim paket berikutnya.
- **Kelemahan Utama:** Utilisasi jaringan sangat buruk! Kapasitas link (bandwidth) yang besar terbuang percuma karena dihabiskan untuk waktu tunggu propagasi sinyal (RTT) yang mendominasi.

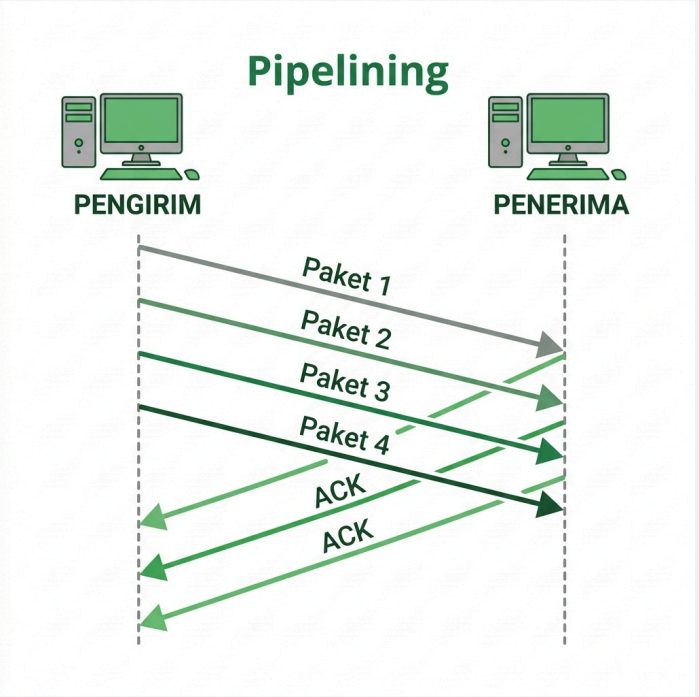


Ilustrasi waktu terbuang saat menunggu paket ACK.

Evolusi RDT & Pipelining (2)

Solusi: Pipelining

- Pengirim diperbolehkan mengirim banyak paket sekaligus secara berturut-turut tanpa harus menunggu ACK dari paket sebelumnya.
- Dengan ini utilisasi link jauh lebih optimal karena pengirim tidak membuang waktu menunggu (gap kosong berkurang drastis).
- Membutuhkan *sequence number* yang lebih besar dan *buffer* (tempat penyimpanan sementara) di pengirim/penerima.

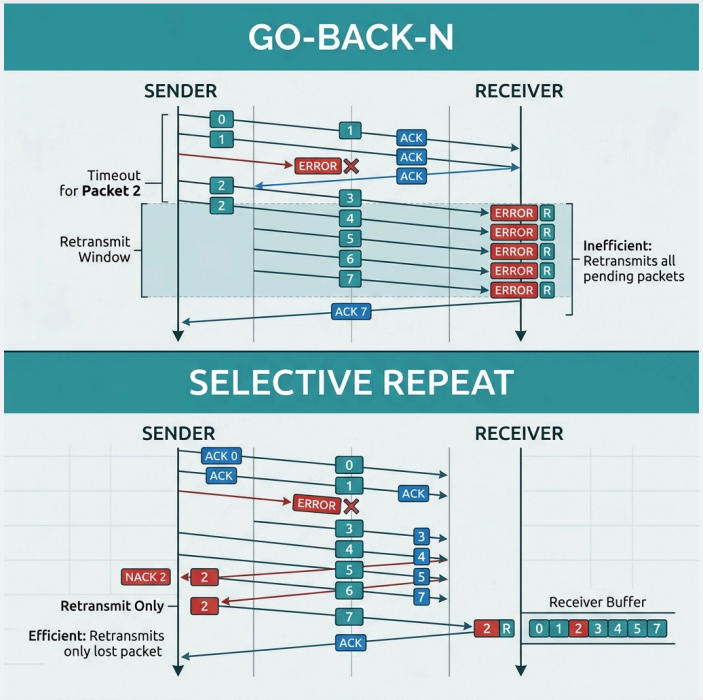


Ilustrasi aliran paket yang dikirimkan tanpa jeda menunggu ACK.

Evolusi RDT & Pipelining (3)

Dua Pendekatan Pipelining

- **Go-Back-N (GBN):**
  - Penerima mengirimkan *Cumulative ACK*.
  - Jika terjadi timeout pada paket n, pengirim mentransmisikan ulang paket n dan **semua** paket setelahnya (Sangat boros bandwidth).
- **Selective Repeat (SR):**
  - Penerima mengirimkan ACK secara individual tiap paket.
  - Penerima mem-buffer paket yang *out-of-order*.
  - Pengirim hanya mentransmisikan ulang paket yang benar-benar hilang/timeout (Lebih efisien).

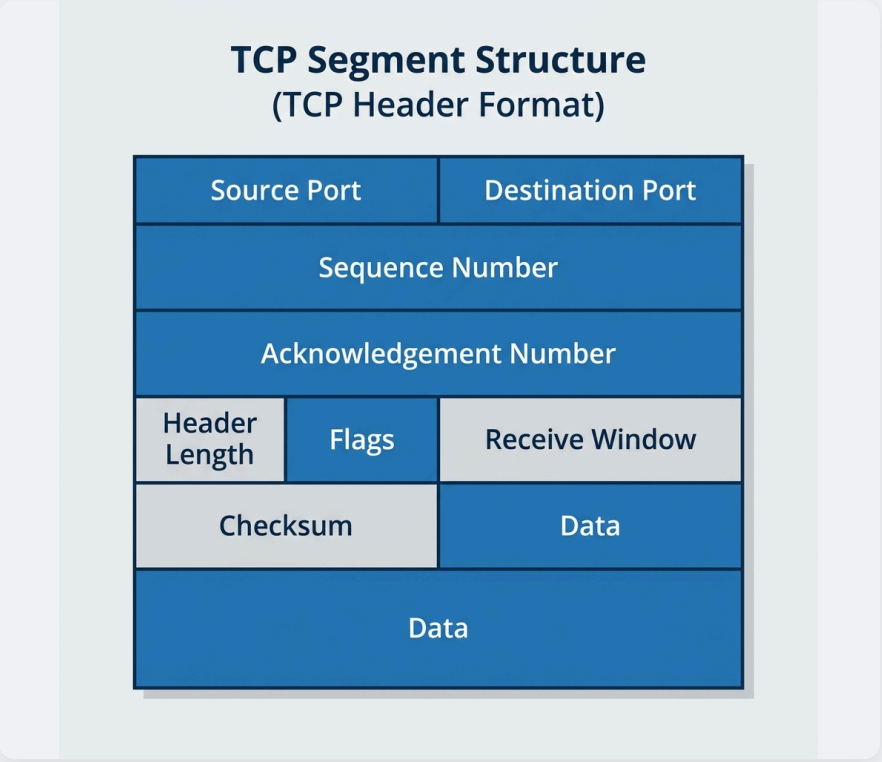


Perbandingan inefisiensi GBN vs efisiensi SR saat terjadi paket error.

5. Transmission Control Protocol (TCP)

Struktur Segmen TCP

- **Source & Dest Port (16-bit):** Untuk multiplexing & demultiplexing.
- **Sequence Number (32-bit):** Nomor urut byte pertama dari data dalam segmen ini (bukan nomor segmen).
- **Acknowledgement Number (32-bit):** Nomor urut byte *selanjutnya* yang diharapkan dari pihak lawan (Cumulative ACK).
- **Header Length (4-bit):** Panjang header (karena ada field Option yang bervariasi).
- **Flags (C, E, U, A, P, R, S, F):**
  - **A (ACK):** Field ACK valid.
  - **S (SYN):** Setup koneksi.
  - **F (FIN):** Tutup koneksi.
- **Receive Window (16-bit):** Untuk *Flow Control* (jumlah byte yang bisa diterima).

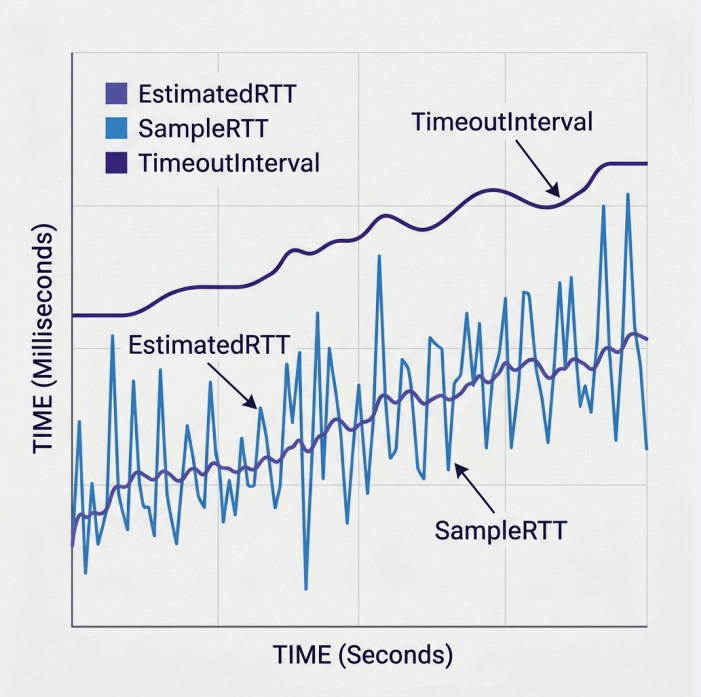


Format Header TCP (Berdasarkan RFC 793)

TCP RTT dan Timeout

Estimasi RTT & Timeout

- TCP mengatur interval *Timeout* secara dinamis karena RTT jaringan selalu berubah.
- Menggunakan `EstimatedRTT` (rata-rata berbobot dari sampel RTT) dan `DevRTT` (margin keamanan/fluktuasi).
- **TimeoutInterval** = EstimatedRTT + 4 * DevRTT.
- Jika terjadi timeout, paket dikirim ulang, dan interval timeout **digandakan**.



Grafik Dinamika SampleRTT, EstimatedRTT, dan Timeout Interval.

Fast Retransmit

Fast Retransmit

- Menunggu timeout seringkali terlalu lama.
- Jika pengirim menerima **3 Duplicate ACKs** untuk data yang sama, ini menandakan paket berikutnya hilang.
- Pengirim akan langsung melakukan **Fast Retransmit** (mengirim ulang paket tersebut) tanpa menunggu timer timeout habis.



Ilustrasi Fast Retransmit: Mengirim ulang sebelum Timeout berdasarkan 3 Duplicate ACKs.

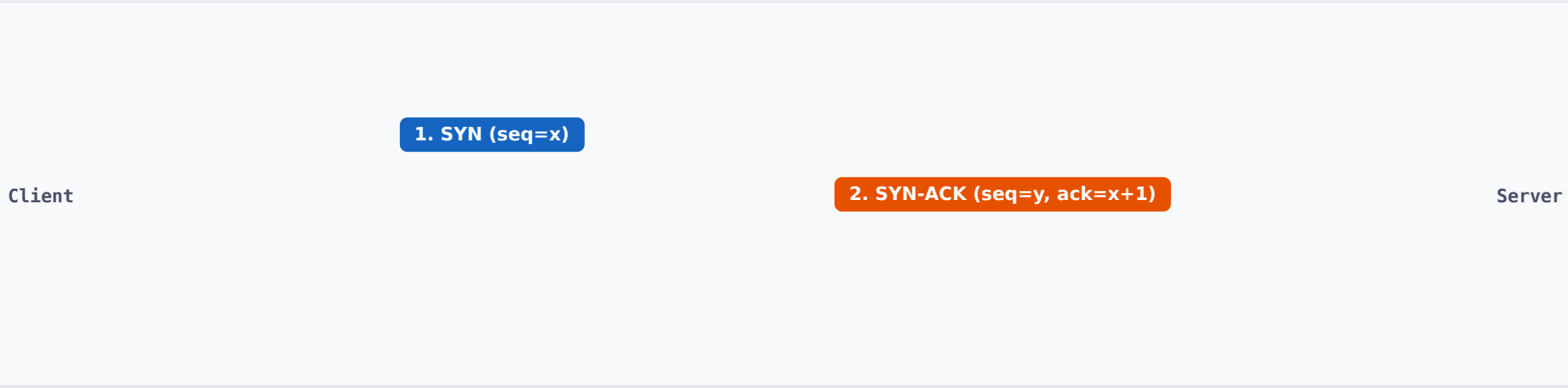
6. Manajemen Koneksi TCP

Manajemen Koneksi (Karakteristik TCP)

- **Point-to-point:** Satu pengirim, satu penerima.
- **Reliable, in-order byte stream:** Data dijamin sampai dengan utuh dan berurutan.
- **Connection-oriented:** Wajib melakukan handshaking sebelum bertukar data.
- **Full duplex data:** Aliran data dua arah (bi-directional) dalam koneksi yang sama.

> Sebelum mempertukarkan data aplikasi, pengirim dan penerima harus saling menyapa (handshake) untuk menyepakati parameter koneksi dan inisialisasi urutan (Sequence Number).

Animasi: TCP 3-Way Handshake



Proses: 1) Klien kirim SYN (meminta koneksi). 2) Server membalas SYN-ACK (setuju & minta balik). 3) Klien membalas ACK (konfirmasi akhir). Koneksi terbentuk!

Menutup Koneksi TCP (Teardown)

Berbeda dengan saat membuka koneksi yang hanya butuh 3 langkah, menutup koneksi TCP butuh **4 langkah** karena sifatnya yang asimetris.

- Koneksi TCP adalah dua jalur komunikasi satu arah yang digabung (Full Duplex).
- Setiap pihak harus menutup jalurnya masing-masing menggunakan bendera **FIN** (Finish).
- Ketika Klien mengirim FIN, ia tidak bisa lagi mengirim data, tapi **masih bisa menerima data** dari Server sampai Server juga mengirim FIN.

Animasi: TCP 4-Way Teardown



Catatan: Terkadang langkah 2 dan 3 digabung menjadi satu paket (FIN+ACK) oleh server jika server memang sudah tidak ada data lagi untuk dikirim.

7. Flow Control (Kendali Aliran)

Flow Control

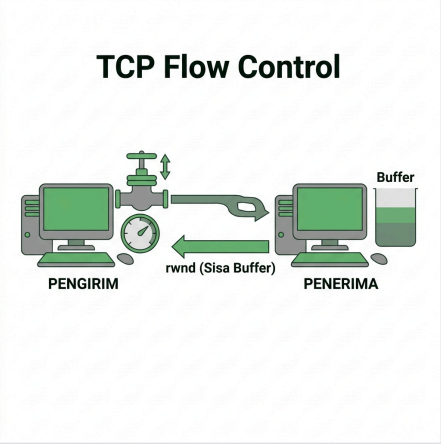
Melindungi Penerima

Masalah: Bagaimana jika aplikasi di sisi penerima sangat lambat membaca data dari buffer TCP, sementara pengirim menembakkan data dengan sangat cepat?

Akibat: Buffer overflow di penerima (data hilang / tertolak).

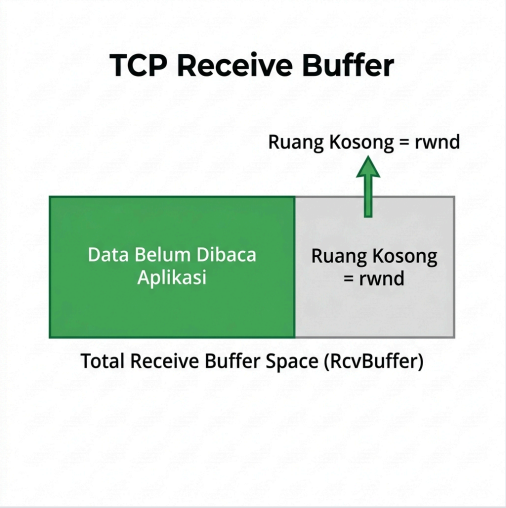
Solusi TCP (Flow Control):

- Penerima secara eksplisit memberitahu pengirim **berapa banyak ruang kosong yang tersisa** di buffernya.
- Nilai ini ditaruh di field **rwnd (Receive Window)** pada header TCP setiap kali penerima membalas dengan ACK.
- Pengirim mematuhi aturan: jumlah data yang sedang melayang (un-ACKed data) *tidak boleh melebihi* nilai `rwnd` saat ini.



Penerima mengatur debit aliran data pengirim melalui parameter rwnd.

Ilustrasi Receive Window (rwnd)



Catatan: Jika aplikasi (di penerima) lambat dalam membaca data dari sistem, maka porsi blok **"Data Belum Dibaca Aplikasi"** akan semakin membesar, yang artinya **"Ruang Kosong (rwnd)"** semakin menyusut. Pengirim akan mengetahui hal ini dari balasan ACK dan akan langsung memperlambat pengiriman secara otomatis untuk mencegah *buffer overflow*.

8. TCP Congestion Control

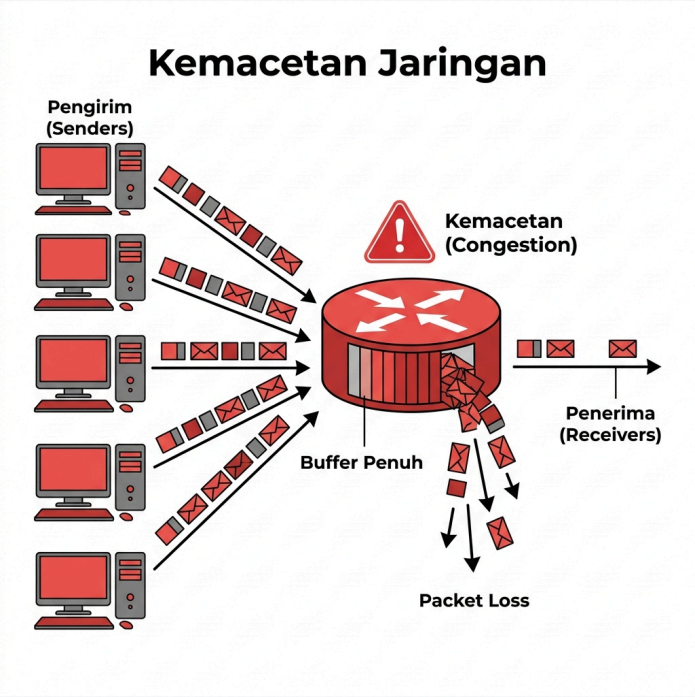
Apa itu Congestion? (Kemacetan Jaringan)

Definisi: Terlalu banyak sumber yang mengirim terlalu banyak data, terlalu cepat, sehingga **jaringan (router di tengah jalan)** tidak sanggup menanganinya.

Ciri-ciri Kemacetan Jaringan:

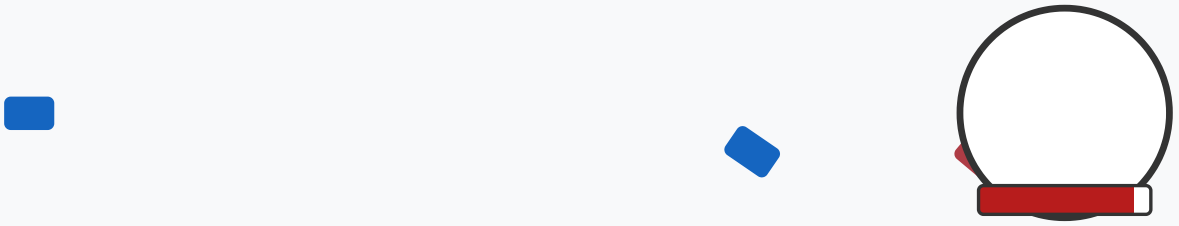
- **Delay Ekstrem:** Paket mengantre lama di dalam buffer router.
- **Packet Loss:** Buffer router penuh, sehingga router terpaksa membuang paket yang baru datang (overflow).

> ⚠️ **Perbedaan Penting:**
> *Flow Control* = Mencegah buffer **Penerima** penuh.
> *Congestion Control* = Mencegah buffer **Router Jaringan** penuh.



Buffer router yang penuh memicu pembuangan paket (Packet Loss).

Animasi: Terjadinya Kemacetan (Congestion)



Buffer Router Penuh → Paket Berikutnya Dibuang (Drop)

Biaya dari kemacetan: Paket yang sudah berhasil melewati setengah jalan internet terbuang percuma, menguras kapasitas jaringan secara global. Pengirim harus melakukan retransmisi.

TCP Congestion Control (AIMD)

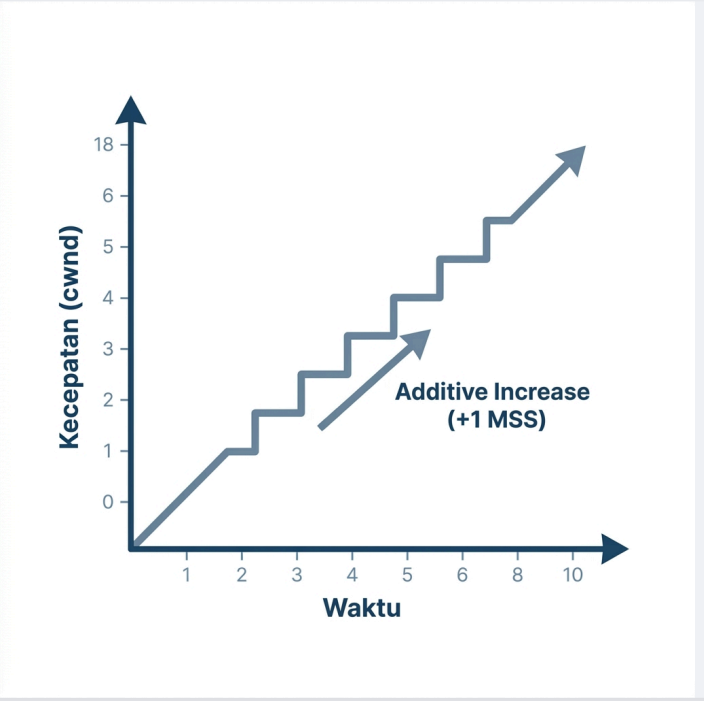
TCP melakukan kontrol kemacetan secara **End-to-End**. TCP tidak bisa bertanya ke router, jadi ia menebak kemacetan dari adanya *packet loss*.

Algoritma AIMD (Additive Increase, Multiplicative Decrease):

- **Additive Increase (Probe Bandwidth):** Selama tidak ada loss, perlahan-lahan tingkatkan kecepatan transmisi (+1 MSS setiap RTT) untuk *meraba-raba* batas bandwidth maksimal.
- **Multiplicative Decrease (Rem Darurat):** Jika terjadi packet loss, segera **potong kecepatan transmisi menjadi setengahnya**.

Algoritma AIMD (1)

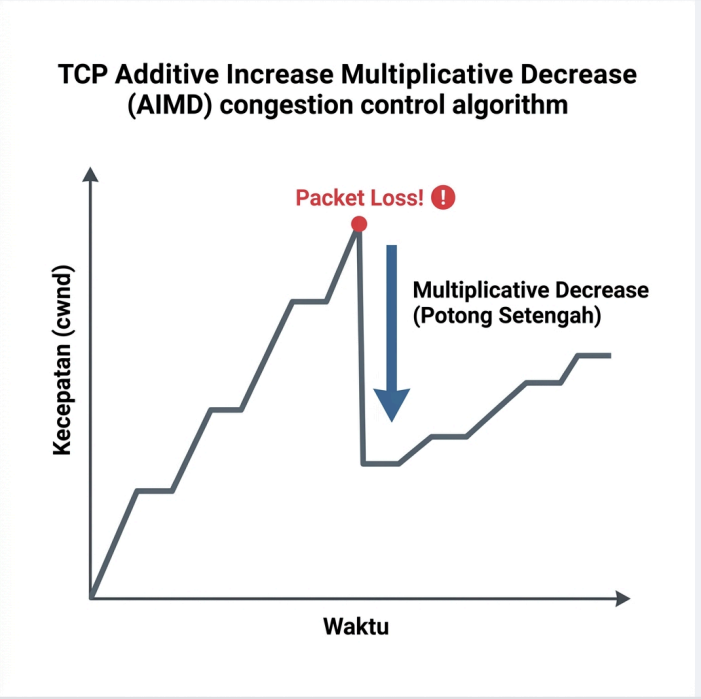
Additive Increase (Probe Bandwidth): Selama tidak ada loss, perlahan-lahan tingkatkan kecepatan transmisi (+1 MSS setiap RTT) untuk *meraba-raba* batas bandwidth maksimal.



Fase Additive Increase (+1 MSS per RTT)

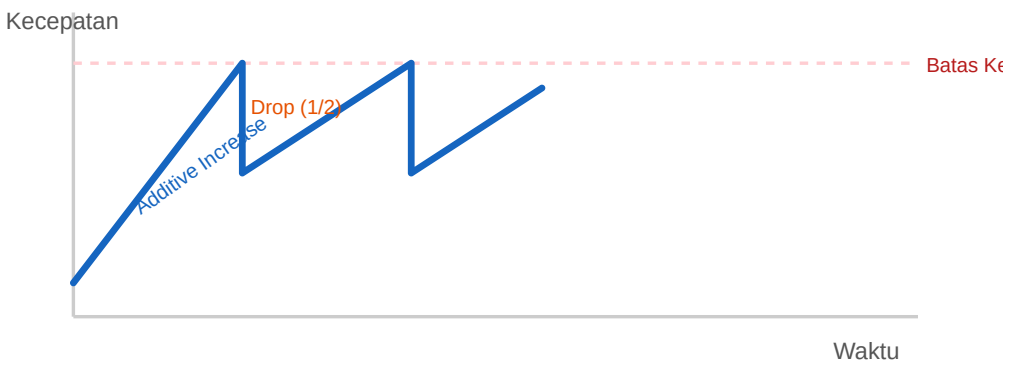
Algoritma AIMD (2)

Multiplicative Decrease (Rem Darurat): Jika terjadi packet loss, segera **potong kecepatan transmisi menjadi setengahnya**.



Fase Multiplicative Decrease (Potong Setengah)

Animasi: Pola Gigi Gergaji (Sawtooth)



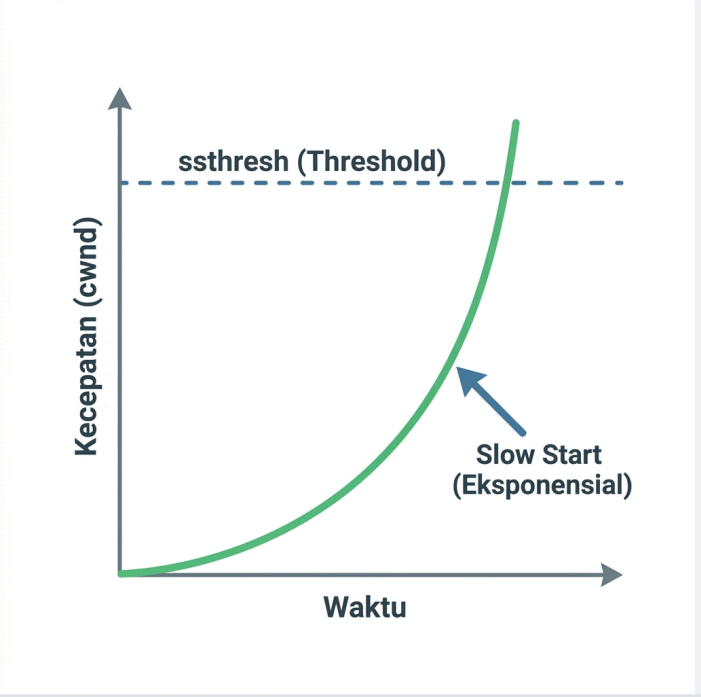
Perilaku ini menghasilkan pola "Gigi Gergaji" yang khas pada grafik throughput TCP. TCP terus "menggoda" batas maksimal jaringan secara berkelanjutan.

Fase TCP

Congestion Control (1)

1. Slow Start

- Saat koneksi baru dimulai, `cwnd` (Congestion Window) diset kecil (misal 1 MSS).
- Namun, TCP ingin menemukan kapasitas maksimum secara cepat.
- Oleh karena itu, `cwnd` ditingkatkan secara **eksponensial** (digandakan setiap RTT) dengan menambahkan 1 MSS untuk setiap ACK yang diterima.
- Fase ini berlangsung hingga `cwnd` mencapai threshold (ssthresh) atau terjadi *loss*.



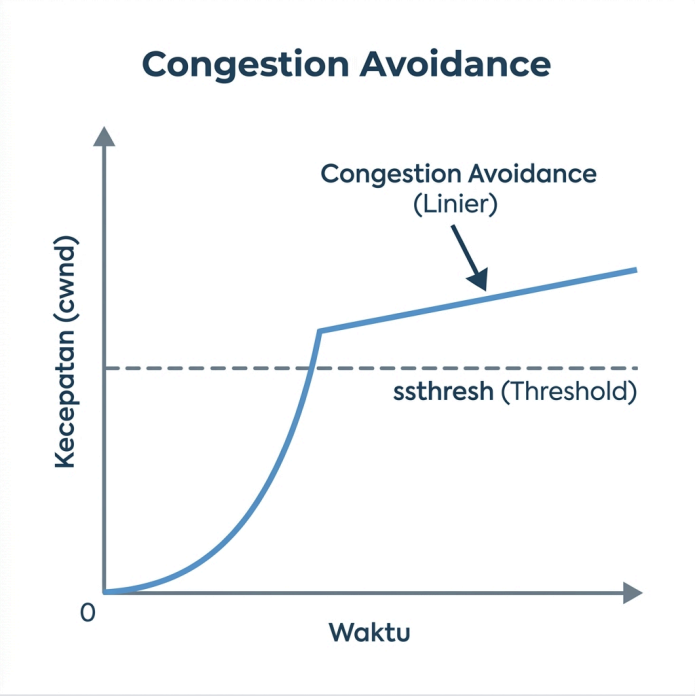
Peningkatan eksponensial menuju batas Threshold

Fase TCP

Congestion Control (2)

2. Congestion Avoidance

- Ketika `cwnd` melewati threshold, peningkatan menjadi linier (**Additive Increase**).
- Tujuannya untuk "meraba" sisa bandwidth dengan hati-hati.



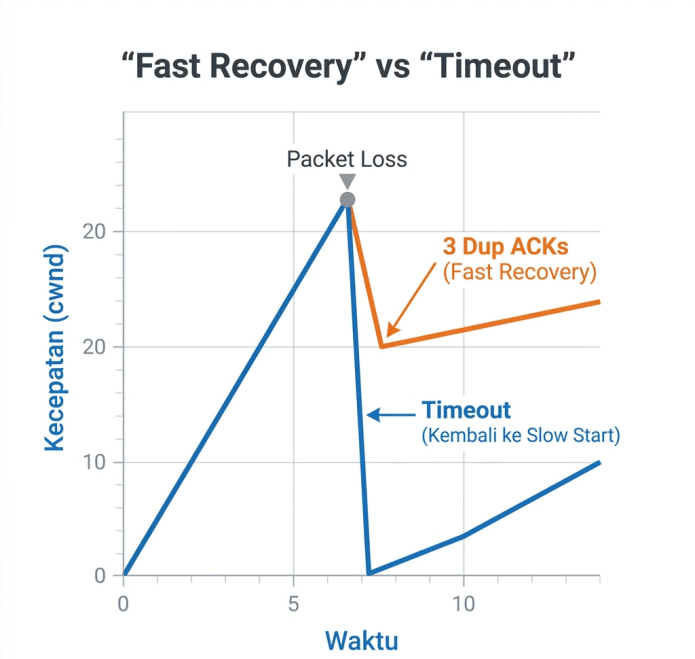
Peningkatan linier setelah melewati Threshold

Fase TCP

Congestion Control (3)

3. Fast Recovery

- Jika terjadi loss akibat **3 Duplicate ACKs**, `cwnd` dipotong menjadi setengahnya, lalu masuk ke fase Fast Recovery.
- Jika loss karena **Timeout**, `cwnd` di-reset kembali ke 1 MSS (kembali ke Slow Start).

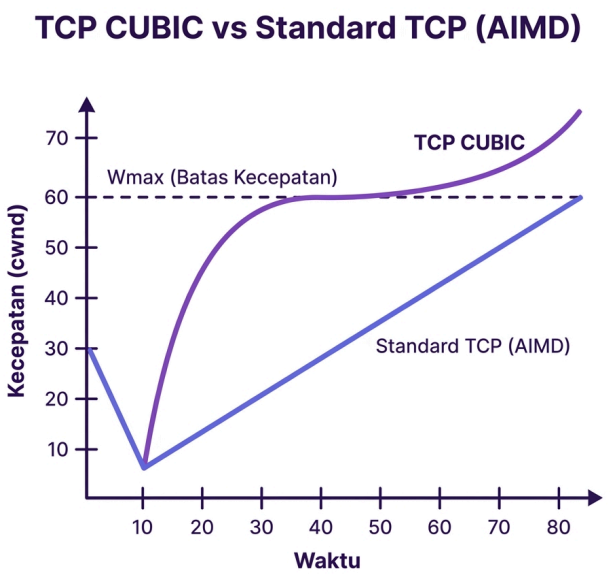


Perbandingan penanganan loss: 3 Dup ACKs vs Timeout

Varian Congestion Control & Fairness (1)

TCP CUBIC

- Varian TCP default pada Linux/Android modern.
- AIMD tradisional kurang agresif di jaringan berkecepatan sangat tinggi.
- **Ide CUBIC:** Menggunakan fungsi kubik (pangkat 3) untuk memulihkan `cwnd` setelah loss.
- Meningkatkan ukuran window dengan sangat cepat di awal, lalu melambat ketika mendekati ukuran maksimum sebelum loss sebelumnya (Wmax).

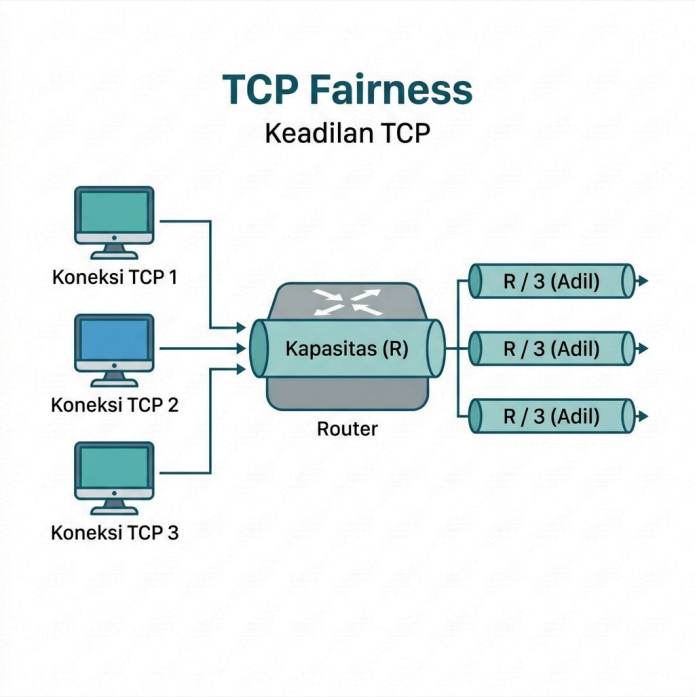


Fungsi Kubik TCP mempercepat pemulihan menuju Wmax

Varian Congestion Control & Fairness (2)

TCP Fairness (Keadilan)

- Jika ada K koneksi TCP yang melewati sebuah bottleneck link berkapasitas R, apakah masing-masing akan mendapat bandwidth sebesar R/K?
- **Ya, TCP pada dasarnya "Fair" (Adil).** Mekanisme AIMD secara matematis memaksa aliran TCP yang bersaing untuk pada akhirnya berbagi bandwidth secara seimbang.
- Namun, koneksi dengan RTT yang lebih kecil (jarak lebih dekat) biasanya bisa meningkatkan `cwnd` lebih cepat, sehingga mengambil porsi lebih banyak.

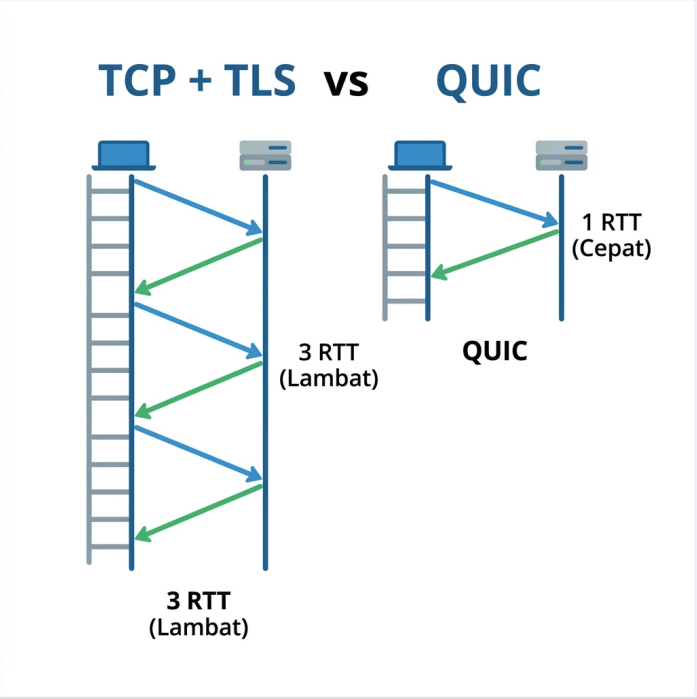


Ilustrasi pembagian Bandwidth (R) secara adil untuk 3 koneksi

9. Evolusi Transport Layer (QUIC)

QUIC: Quick UDP Internet Connections

- Dirancang oleh Google, kini menjadi fondasi **HTTP/3**.
- **Menggunakan UDP** di bawahnya, tetapi mengimplementasikan keandalan (RDT), congestion control, dan keamanan (TLS) di level aplikasi/QUIC itu sendiri.
- **Kelebihan Utama:**
  - **Fast Connection Setup:** Menggabungkan handshake transport dan kriptografi (TLS) menjadi 1 RTT (atau bahkan 0 RTT!). TCP+TLS biasanya butuh 2-3 RTT.
  - **No Head-of-Line Blocking:** Memiliki konsep "Streams" independen. Jika satu paket hilang di Stream A, Stream B tetap bisa diproses tanpa terhenti.
- Masa depan protokol web yang lebih cepat dan efisien!



QUIC (1 RTT) vs TCP+TLS (3 RTT)

Rangkuman Pertemuan 5

Transport Layer, UDP & RDT

- **Transport Layer:** Komunikasi antar proses (Process-to-Process) menggunakan Port Numbers. Melakukan proses Multiplexing/Demultiplexing.
- **UDP:** Protokol connectionless, unreliable, namun cepat tanpa overhead. Sangat ideal untuk layanan streaming dan DNS.
- **RDT & Pipelining:** Mekanisme keandalan transfer data menggunakan Checksum, ACK/NACK, Sequence Number, dan Timer.
- **Pipelining (GBN & SR):** Protokol yang mengizinkan pengiriman banyak paket sekaligus tanpa menunggu ACK untuk memaksimalkan utilitas bandwidth.

TCP, Congestion Control & QUIC

- **TCP:** Protokol connection-oriented, andal (reliable), dan berurutan (in-order byte-stream).
- **Manajemen Koneksi:** 3-Way Handshake (setup) & 4-Way Teardown (teardown).
- **Flow Control:** Mekanisme untuk melindungi buffer *penerima* agar tidak kewalahan (menggunakan `rwnd`).
- **Congestion Control:** Melindungi antrean *router* jaringan agar tidak macet menggunakan algoritma AIMD (Slow Start, Congestion Avoidance, Fast Recovery).
- **QUIC:** Evolusi Transport Layer untuk HTTP/3 yang dibangun di atas UDP guna menghilangkan efek *Head-of-Line Blocking*.

📝 Tugas Individu — Pertemuan 5

Dikerjakan analitik, ditulis tangan di kertas folio, difoto, dikirim ke e-learning.

1. **[Analogi Kurir Pos]** Gunakan Analogi lapisan Transport sebagai "Petugas Pos" yang menyortir surat (port/soket). Jika Petugas Pos mempekerjakan dua kurir pembantu: "Si Cepat-tapi-Ceroboh" dan "Si Lambat-tapi-Pasti", petakan karakteristik kedua kurir tersebut ke dalam mekanisme *multiplexing*, pengurutan, dan keandalan (reliability) milik protokol **UDP** dan **TCP**! Jelaskan mengapa keduanya tetap dibutuhkan!

2. **[Studi Kasus Grafis TCP]** Gambarkan secara mandiri di kertas Anda grafik gigi gergaji (AIMD) TCP dengan skenario berikut: Koneksi dimulai dengan `ssthresh` = 8 MSS. Siklus ke-1 hingga 4 berjalan normal. Pada awal Siklus ke-5, terjadi *packet loss* akibat **3 Duplicate ACKs**. Koneksi pulih dan berjalan normal dari Siklus ke-6 hingga ke-8. **Berapakah nilai pasti `cwnd` (Congestion Window) pada Siklus ke-4, Siklus ke-5 (saat potong), dan Siklus ke-8?** (Petunjuk: Pahami perbedaan Slow Start vs Congestion Avoidance).

3. **[Analisis QUIC & SIAKAD]** Protokol QUIC (HTTP/3) berevolusi di atas UDP untuk memecahkan masalah *Head-of-Line Blocking* milik TCP. Jika kelak SIAKAD ITERA dimigrasikan ke HTTP/3, jelaskan sebuah skenario spesifik saat masa KRS-an (jaringan sibuk) bagaimana arsitektur QUIC mencegah *packet loss* pada satu *request* (misal: *request* gambar aset web) memblokir antrean *request* krusial lainnya (misal: *request* simpan KRS)!

Tanya Jawab & Diskusi

Pertemuan berikutnya: **IP Addressing & Subnetting**

Referensi mandiri: Kurose & Ross Chapter 3 (Transport Layer)